

# Caderno de Validação Manual dos Requisitos

SolidaryTech: como comprovar, passo a passo e em execução, que cada exigência do Hackathon está completa e funcionando no AWS Academy.

---

39 verificações rastreáveis ao enunciado · comando exato · resultado esperado · o que fazer se falhar · campo de aceite

## PROJETO

**SolidaryTech: plataforma de doações para ONGs**

## AMBIENTE DE REFERÊNCIA

**AWS Academy Learner Lab · conta 716532857874 · us-east-1**

## INTEGRANTES

**Leonardo Alves Freitas (rm369434)  
Antonio Eduardo Silveira Demarchi (rm370045)**

## VERSÃO · DATA

**1.1 · 25/09/2026**

## ÚLTIMA EXECUÇÃO COMPLETA

**25/09/2026 12:03 UTC · 48/48 verificações OK**

## REPOSITÓRIO

**[github.com/DemarchiWorking/fiap-tc-5-solidarytech](https://github.com/DemarchiWorking/fiap-tc-5-solidarytech)**

# 00 Como usar este caderno

Leia antes de executar qualquer passo

Cada exigência do enunciado virou um **cartão de requisito** com: o texto do enunciado, onde executar, os passos com o comando exato, o resultado esperado (o valor medido em 25/09) e o que fazer se não bater. Ao final de cada cartão há um campo de aceite para o responsável marcar.

|                                                         |                                                       |                                                   |                                               |
|---------------------------------------------------------|-------------------------------------------------------|---------------------------------------------------|-----------------------------------------------|
| <b>48/48</b><br>verificações automáticas OK<br>em 25/09 | <b>15/15</b><br>Applications ArgoCD<br>Synced/Healthy | <b>0 %</b><br>falha em 12.879 requisições<br>(k6) | <b>0</b><br>recursos IAM criados<br>(Academy) |
|---------------------------------------------------------|-------------------------------------------------------|---------------------------------------------------|-----------------------------------------------|

## Legenda

| Marcador          | Significado                                                                                                                                                             |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| TERMINAL          | Terminal do <b>WSL (Ubuntu)</b> , já dentro da pasta do projeto ( <code>tc5</code> ). Nunca CMD ou PowerShell: o <code>kubect1</code> do Windows não conhece o cluster. |
| NAVEGADOR         | Abrir no navegador. <code>\$BASE</code> é o endereço público do NLB (passo P-02).                                                                                       |
| CONSOLE AWS       | Painel do AWS Academy → <b>Start Lab</b> → clicar em <b>AWS</b> (bolinha verde).                                                                                        |
| REPOSITÓRIO       | Leitura de arquivo do repositório: prova documental, não de execução.                                                                                                   |
| MEDIDO            | Validado em execução em 25/09/2026; o valor esperado é o medido.                                                                                                        |
| DOCUMENTAL        | Exigência atendida por documento (PCN, forecast, desenho): valide lendo.                                                                                                |
| PENDENTE DO GRUPO | Depende de conta ou ação do grupo (Datadog, prints, vídeo).                                                                                                             |

## Regras de segurança durante a validação

- **Nenhuma credencial em arquivo, print ou chat.** Senhas saem só no seu terminal ( `./solidary senhas` ). A chave do Datadog vive no AWS Secrets Manager e **não** é digitada em nenhum passo deste caderno.
- Passos marcados com **faixa vermelha** alteram o ambiente de propósito (falha simulada, restore). Execute exatamente como descrito; todos se desfazem sozinhos ou têm o passo de volta.
- Não use o endereço `*.eks.amazonaws.com` no navegador: é a API do Kubernetes e responde **401** por projeto. A interface é pelo NLB.
- A sessão do Learner Lab dura 4 horas. Se expirar, refaça P-01 e P-02: **os endereços mudam**.

## Controle do documento

| Versão | Data (UTC)       | Alteração                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|--------|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1.0    | 25/09/2026 12:20 | Criação: 39 cartões e matriz de rastreabilidade. Todos os comandos executados na conta 716532857874 antes de entrar no documento.                                                                                                                                                                                                                                                                                                                                                                  |
| 1.1    | 25/09/2026 13:45 | Correções após nova validação em execução: horário da última validação completa (12:03, não 10:50); log do worker lido em <b>todas</b> as réplicas (F0-10); prova de selfHeal por drift no HPA, porque o teste antigo media o <code>minReplicas</code> (F0-07); elasticidade 1 → 6 → 1 comprovada após as correções <code>af8a695</code> e <code>e03d991</code> (F0-11); acesso ao ArgoCD e ao Grafana detalhado, com login recomendado por túnel cifrado (P-03); aviso do Chrome para HTTP (A.1). |

**Classificação:** uso do grupo e da banca avaliadora. Não contém senha, token nem chave: credenciais são lidas no terminal de quem valida. **Responsáveis:** os integrantes da capa. **Fonte:** [docs/relatorio/VALIDACAO-MANUAL.html](#) no repositório (o PDF é gerado a partir dele).

Sumário

|           |                                  |           |                                         |
|-----------|----------------------------------|-----------|-----------------------------------------|
| <b>P</b>  | Preparação da sessão de teste    | <b>F3</b> | ITSM e AIOps                            |
| <b>RO</b> | Regra de Ouro: projeto evolutivo | <b>F4</b> | Segurança, Backup e DR                  |
| <b>F0</b> | Fundação DevOps (obrigatória)    | <b>E</b>  | Entregáveis e evidências visuais        |
| <b>F1</b> | SRE: SLI, SLO e MTTR             | <b>Z</b>  | Encerramento e higiene                  |
| <b>F2</b> | FinOps                           | <b>A</b>  | Apêndices: diagnóstico, acessos, aceite |

# 01 Matriz de rastreabilidade

Exigência do enunciado → verificação deste caderno  
→ situação em 25/09

| ID    | Exigência                                     | Como é comprovada                       | Situação          |
|-------|-----------------------------------------------|-----------------------------------------|-------------------|
| RO-01 | Sem deploy manual via kubectl                 | Commits do bot + ArgoCD                 | MEDIDO            |
| RO-02 | Sem infraestrutura clicada no console         | terraform plan sem mudanças             | MEDIDO            |
| RO-03 | Sem voo cego                                  | F0-08 + F0-09                           | MEDIDO            |
| F0-01 | Dockerfiles otimizados (3 serviços)           | Multi-stage, non-root, 0 HIGH/CRITICAL  | MEDIDO            |
| F0-02 | Kubernetes gerenciado (EKS)                   | EKS 1.34 ACTIVE, 3 nós                  | MEDIDO            |
| F0-03 | Terraform: cluster, bancos, mensageria, rede  | 52 recursos no state                    | MEDIDO            |
| F0-04 | Pipelines com testes                          | Jobs Lint e Testes verdes               | MEDIDO            |
| F0-05 | SAST/SCA (Trivy/Sonar)                        | gosec/bandit + Trivy + SBOM             | MEDIDO            |
| F0-06 | Construção e publicação da imagem             | ECR imutável = imagem em execução       | MEDIDO            |
| F0-07 | GitOps com ArgoCD                             | 15/15 + selfHeal em 18 s                | MEDIDO            |
| F0-08 | Prometheus, Grafana, Loki, OpenTelemetry      | 27 alvos, 0 down                        | MEDIDO            |
| F0-09 | APM com Distributed Tracing                   | Datadog: 0 × 403, trace via SQS         | MEDIDO            |
| F0-10 | Aplicações funcionais (3 APIs + fila)         | 11 chamadas públicas                    | MEDIDO            |
| F0-11 | Elasticidade (HPA)                            | Worker 1 → 6 → 1                        | MEDIDO            |
| F1-01 | ≥ 2 SLIs do donation-service (Golden Metrics) | 3 recording rules                       | MEDIDO            |
| F1-02 | SLO para cada SLI                             | 99,9 % · 99 % < 300 ms · 99,5 % < 60 s  | DOCUMENTAL        |
| F1-03 | Dashboard exclusivo de SLO e error budget     | Painel Grafana SRE                      | MEDIDO            |
| F1-04 | Evidenciar redução de MTTR                    | Chaos drill (MTTD 76 s)                 | MEDIDO            |
| F2-01 | Tags Project / Environment / CostCenter       | Mesmo total nas 3 tags                  | MEDIDO            |
| F2-02 | Rightsizing via GitOps                        | requests/limits em 100 % dos containers | MEDIDO            |
| F2-03 | Forecast mensal                               | US\$ 202,74/mês                         | DOCUMENTAL        |
| F2-04 | ≥ 1 recomendação nativa de nuvem              | 5 recomendações quantificadas           | DOCUMENTAL        |
| F3-01 | IA do APM (Watchdog) detectando anomalias     | Monitor Watchdog                        | PENDENTE DO GRUPO |
| F3-02 | Ciclo de vida do incidente                    | Diagrama + post-mortem                  | DOCUMENTAL        |
| F4-01 | PCN com RTO e RPO                             | RTO 1 h · RPO 15 min + RDS PITR         | MEDIDO            |

| ID    | Exigência                                                | Como é comprovada                 | Situação          |
|-------|----------------------------------------------------------|-----------------------------------|-------------------|
| F4-02 | Velero: backup de manifestos e volumes em bucket externo | 914 itens, bucket em us-west-2    | MEDIDO            |
| F4-03 | Restore comprovado                                       | PVC Bound em 12 s                 | MEDIDO            |
| F4-04 | Ambiente espelho em outra região                         | Plano 34/0/0 em us-west-2         | MEDIDO            |
| F4-05 | Segurança de rede e IMDS                                 | NetworkPolicies + IMDS bloqueado  | MEDIDO            |
| F4-06 | Gestão de segredos                                       | Secrets Manager, 0 segredo no Git | MEDIDO            |
| F4-07 | Conformidade com o AWS Academy                           | 16 verificações + 0 IAM           | MEDIDO            |
| E-01  | Repositório público com IaC, manifestos e pipelines      | Aba anônima                       | MEDIDO            |
| E-02  | Vídeo de até 20 minutos                                  | Roteiro pronto                    | PENDENTE DO GRUPO |
| E-03  | PDF com nomes, RMs, links e evidências visuais           | 10 prints + link do vídeo         | PENDENTE DO GRUPO |

## P-01 Credenciais da sessão e pré-voo

MEDIDO

Onde:   Tempo: 3 min

- 1 No painel do AWS Academy: **Start Lab** → aguarde a bolinha verde → **AWS Details** → **AWS CLI: Show**. Copie o bloco.
- 2 Abra o WSL e entre no projeto:

```
wsl  
tc5
```

- 3 Cole o bloco em `~/.aws/credentials` (substituindo o anterior) e rode o pré-voo:

```
nano ~/.aws/credentials      # colar, Ctrl+O, Enter, Ctrl+X  
sed -i 's/\r$//' ~/.aws/credentials # remove CRLF, se colado do Windows  
./solidary pre-voo
```

### RESULTADO ESPERADO

Veredito **GO**, com a linha "credencial do Datadog no cofre (...d0e9 @ datadoghq.com)" e a conta da sessão igual à do GitOps.

### SE NÃO BATER

**NO-GO por conta diferente:** o laboratório trocou de conta. Siga o [COMO-SUBIR.md](#) (bootstrap → apply → configurar-repo → deploy, ~35 min). **ExpiredToken:** sessão vencida; repita o passo 1.

☐ Validado Responsável \_\_\_\_\_ Data/hora \_\_\_\_\_

## P-02 Conectar ao cluster e descobrir o endereço público

MEDIDO

Onde:  Tempo: 1 min

- 1 Aponte o `kubectl` para o EKS e confira os nós:

```
./solidary kubeconfig  
kubectl get nodes
```

- 2 Guarde o endereço público numa variável; todos os cartões usam `$BASE`:

```
BASE="http://${kubectl -n ingress-nginx get svc ingress-nginx-controller -o  
jsonpath='{.status.loadBalancer.ingress[0].hostname}'}"  
echo $BASE
```

### RESULTADO ESPERADO

3 nós Ready em v1.34. `$BASE` no formato `http://...elb.us-east-1.amazonaws.com`.

### SE NÃO BATER

`dial tcp [::1]:8080`: você está no PowerShell/CMD, não no WSL. `$BASE` vazio: o ingress ainda não recebeu o NLB; aguarde 3 min.

☐ Validado Responsável \_\_\_\_\_ Data/hora \_\_\_\_\_

Onde: **TERMINAL** Tempo: 1 min

- 1 Exiba usuário, senha e URL base (somente no seu terminal):

```
./solidary senhas
```

- 2 Se o atalho falhar, leia cada senha direto do Secret de origem:

```
kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath='{.data.password}' |  
base64 -d; echo  
kubectl -n monitoring get secret grafana-admin -o jsonpath='{.data.admin-password}' | base64  
-d; echo
```

- 3 **Login recomendado: por túnel cifrado.** O ingress é HTTP (o Learner Lab não oferece domínio para TLS), e a senha do **admin** trafegaria aberta pela internet. O **port-forward** passa pela API do Kubernetes, que é TLS. Um terminal para cada, aberto enquanto usar a interface:

```
# ArgoCD → http://localhost:8080/argocd/  
kubectl -n argocd port-forward svc/argocd-server 8080:80  
# Grafana → http://localhost:3000/grafana/  
kubectl -n monitoring port-forward svc/kube-prometheus-stack-grafana 3000:80
```

- 4 Alternativa, só em rede confiável: **\$BASE/argocd/** e **\$BASE/grafana/** pelo NLB. Nesse caso o Chrome mostra "O site não pode fazer uma conexão segura": confira que o endereço é o **\$BASE** e clique em **Ir para o site**.

**RESULTADO ESPERADO**

Usuário **admin** nos dois, **sem cadastro**. As senhas não têm valor padrão: são geradas aleatoriamente na instalação (ArgoCD: Secret **argocd-initial-admin-secret** ; Grafana: Secret **grafana-admin** ) e mudam a cada ambiente recriado. O login leva às telas de Applications e de Dashboards.

**Segurança:** não cole as senhas em chat, documento ou print. Elas morrem com o ambiente.

**Em produção fora do laboratório**, o acesso seria diferente, e o relatório declara o risco residual na seção *Revisão de segurança da superfície exposta*: TLS no ingress (domínio e certificado, que o Learner Lab não oferece); login por SSO/OIDC em vez do **admin** local; e, no primeiro acesso, troca da senha inicial do ArgoCD (*User Info* → *Update Password*) e remoção do Secret **argocd-initial-admin-secret**, como recomenda a documentação do ArgoCD.

☐ Validado Responsável \_\_\_\_\_ Data/hora \_\_\_\_\_

Onde: TERMINAL Tempo: 6 min

- 1 Dispare o teste de carga k6 (roda dentro do cluster, via GitOps):

```
./solidary carga  
kubectl -n solidary-loadtest get jobs -w      # Ctrl+C quando COMPLETIONS = 1/1
```

- 2 Leia o resumo do k6:

```
kubectl -n solidary-loadtest logs job/${kubectl -n solidary-loadtest get jobs --sort-  
by=.metadata.creationTimestamp -o jsonpath='{.items[-1:].metadata.name}'} | grep -E  
"http_req_failed|http_req_duration|checks"
```

**RESULTADO ESPERADO**

checks 100% · http\_req\_failed 0.00% · p95 na casa de **7 ms** (25/09: 12.879 requisições, p95 7,2 ms).

Sem carga, os painéis de SLO mostram `NaN`, o que é o comportamento correto sem tráfego, e o Watchdog não tem linha de base. **Rode sempre antes de prints e do vídeo.**

☐ Validado Responsável \_\_\_\_\_ Data/hora \_\_\_\_\_

## RO-01 Nenhum deploy manual via kubectl

MEDIDO

"Não haverá deploy manual via kubectl."

Onde:   Tempo: 3 min

- 1 Mostre que toda versão em produção nasceu de um commit do bot da CI:

```
git pull --ff-only
git log --oneline --grep="^deploy(" -5
```

- 2 Compare a tag do último commit com a imagem que está rodando:

```
kubectl -n solidarity-donation get deploy donation-service -o
jsonpath='{.spec.template.spec.containers[0].image}{"\n"}'
```

- 3 No ArgoCD ( `$BASE/argocd/` ) abra **app-donation** → **History and rollback**: a revisão implantada é o mesmo commit.

### RESULTADO ESPERADO

Commits `deploy(donation-service): <sha>` de autoria **github-actions[bot]**; a imagem em execução termina com o mesmo `<sha>` (25/09: `b02dc80...` ).

☐ Validado Responsável \_\_\_\_\_ Data/hora \_\_\_\_\_

## RO-02 Nenhuma infraestrutura criada no console

MEDIDO

"Não haverá infraestrutura clicada no console."

Onde:  Tempo: 3 min

- 1 Rode um plano (somente leitura) contra a nuvem real:

```
./solidary plan
```

### RESULTADO ESPERADO

`No changes. Your infrastructure matches the configuration.` : o que existe na AWS é exatamente o código. A mesma verificação roda na CI (workflow *Terraform*, ação `plan` ).

### SE NÃO BATER

Se aparecer diferença, alguém alterou recurso pelo console (drift). Não aplique às cegas: leia o plano e reverta pelo código.

☐ Validado Responsável \_\_\_\_\_ Data/hora \_\_\_\_\_

**R0-03****Nenhum voo cego: monitoramento profundo****MEDIDO**

"Não haverá voo cego sem monitoramento profundo."

Comprovado pela soma de **F0-08** (métricas e logs), **F0-09** (traces no APM) e **F1-03** (SLOs). Marque este cartão após concluir os três.

☐ Validado    Responsável \_\_\_\_\_    Data/hora \_\_\_\_\_

## F0-01 Dockerfiles otimizados para os três serviços

MEDIDO

Onde: **TERMINAL** **REPOSITÓRIO** Tempo: 3 min Evidência: **07-evidencias/devsecops-auditoria.txt**

- 1 Conte os estágios (multi-stage) e confira o usuário não-root e a imagem final:

```
for s in ngo donation volunteer; do f=services/$s-service/Dockerfile; echo "$s:
estagios=$(grep -c '^FROM' $f) | $(grep -E '^USER' $f | tail -1) | $(grep '^FROM' $f | tail
-1)"; done
```

- 2 Leia o resultado do scanner sobre as imagens publicadas:

```
grep -iE "HIGH|CRITICAL|Total" docs/07-evidencias/devsecops-auditoria.txt | head -15
```

### RESULTADO ESPERADO

ngo e volunteer: 3 estágios, **USER 10001:10001**, runtime **python:3.12-slim-bookworm** sem **pip**. donation: 4 estágios, **USER 65532:65532**, runtime **distroless static-debian12:nonroot**. **0 HIGH/CRITICAL** nas três imagens.

☐ Validado Responsável \_\_\_\_\_ Data/hora \_\_\_\_\_

## F0-02 Implantação em Kubernetes gerenciado (EKS)

MEDIDO

Onde: **TERMINAL** **CONSOLE AWS** Tempo: 2 min Print: **f0-pods-running.png**

- 1 Estado do cluster gerenciado:

```
aws eks describe-cluster --name solidaritytech-prod-eks --query "cluster.[version,status]" --
output text
```

- 2 Pods das aplicações (este é o print **f0-pods-running.png**):

```
kubectl get pods -A | grep solidarity
```

- 3 Opcional: Console AWS → **EKS** → **Clusters** → **solidaritytech-prod-eks**.

### RESULTADO ESPERADO

**1.34 ACTIVE**. Todos os pods **Running** com **0 restarts**: 2× ngo, 2× donation, 2× volunteer-service e o worker (1 a 6 conforme a fila). Jobs **db-init** e **k6-run** aparecem como **Completed**, o que é normal.

No console, o EKS pode avisar que a LabRole não vê recursos do Kubernetes. É limitação do Learner Lab; o **kubectl** vê tudo.

☐ Validado Responsável \_\_\_\_\_ Data/hora \_\_\_\_\_

**F0-03 Terraform provisionando cluster, bancos, mensageria e rede****MEDIDO**Onde: **TERMINAL** Tempo: 2 min Código: **infra/modules/\***

- 1 Total de recursos sob gestão do Terraform:

```
terraform -chdir=infra/environments/prod-use1 state list | wc -l
```

- 2 Um recurso de cada categoria exigida:

```
terraform -chdir=infra/environments/prod-use1 state list | grep -E  
"aws_eks_cluster|aws_db_instance|aws_dynamodb_table|aws_sqs_queue|aws_vpc\.|aws_subnet"
```

**RESULTADO ESPERADO**

**52** recursos. Cluster: **aws\_eks\_cluster.principal** ; bancos: **aws\_db\_instance** (RDS PostgreSQL) e **aws\_dynamodb\_table** ; mensageria: **aws\_sqs\_queue** principal e DLQ; rede: VPC e 4 subnets.

☐ Validado Responsável \_\_\_\_\_ Data/hora \_\_\_\_\_**F0-04 Pipelines de CI com testes****MEDIDO**Onde: **NAVEGADOR** Tempo: 3 min Print: **f0-pipeline-verde.png**

- 1 Abra [github.com/DemarchiWorking/fiap-tc-5-solidarytech/actions](https://github.com/DemarchiWorking/fiap-tc-5-solidarytech/actions) → à esquerda, **CI — donation-service** → execução mais recente.
- 2 Confira os cinco jobs em sequência e abra o job **Testes**.

**RESULTADO ESPERADO**

Verdes: **Lint** → **Testes** (Go com **-race** ; pytest nos serviços Python) → **SAST** → **Build, scan e push** → **Atualizar o GitOps**. Mesmo padrão para ngo e volunteer.

**SE NÃO BATER**

Para disparar de novo: clique no nome do workflow na barra lateral → **Run workflow**. Os secrets AWS do GitHub expiram com a sessão do lab: antes, rode **./solidary sync-creds**.

☐ Validado Responsável \_\_\_\_\_ Data/hora \_\_\_\_\_

**F0-05 Segurança na esteira: SAST e SCA****MEDIDO**

"Pipelines com testes, SAST/SCA (Trivy/Sonar)..."

Onde: **NAVEGADOR** Tempo: 3 min

- 1 Na mesma execução, abra **SAST (gosec/bandit + SonarCloud)**: análise estática do código.
- 2 Abra **Build, scan e push**: Trivy em duas camadas (dependências e imagem), com severidade CRITICAL bloqueante, e geração de SBOM.
- 3 Na página da execução, role até **Artifacts**: o SBOM está anexado.

**RESULTADO ESPERADO**

SAST sem achados bloqueantes; Trivy com **0 HIGH/CRITICAL**; artefato de SBOM presente. O histórico do repositório registra 4 CVEs reais barradas pelo gate (ver [docs/10](#), passo 8.3).

☐ Validado Responsável \_\_\_\_\_ Data/hora \_\_\_\_\_

**F0-06 Construção e publicação da imagem****MEDIDO**

Onde: **TERMINAL** Tempo: 2 min

- 1 Repositórios imutáveis e com scan no push:

```
aws ecr describe-repositories --query "repositories[?starts_with(repositoryName, 'solidarytech/')].repositoryName,imageTagMutability,imageScanningConfiguration.scanOnPush]" --output table
```

- 2 Última imagem publicada pela CI:

```
aws ecr describe-images --repository-name solidarytech/donation-service --query 'sort_by(imageDetails,&imagePushedAt)[-1].[imageTags[0],imagePushedAt]' --output text
```

**RESULTADO ESPERADO**

3 repositórios **IMMUTABLE** com scan **True**. A tag é o SHA de 40 caracteres do commit, o mesmo visto em RO-01.

☐ Validado Responsável \_\_\_\_\_ Data/hora \_\_\_\_\_

**F0-07** GitOps com ArgoCD, incluindo a autocorreção

MEDIDO

Onde: **TERMINAL** **NAVEGADOR** Tempo: 4 min Print: `f0-argocd.png`

- 1 Estado de todas as aplicações:

```
kubectl -n argocd get applications
```

- 2 Abra `$BASE/argocd/`, entre com `admin` e a senha do P-03. A tela inicial é o print `f0-argocd.png`.

- 3 **Prova de GitOps:** altere na mão um campo que o Git declara (o teto do HPA) e veja o ArgoCD desfazer:

```
kubectl -n solidary-ngo patch hpa ngo-service --type merge -p '{"spec":{"maxReplicas":3}}'
kubectl -n solidary-ngo get hpa ngo-service -w # Ctrl+C quando MAXPODS voltar a 6
```

**RESULTADO ESPERADO**

15 Applications **Synced** / **Healthy**. O **MAXPODS** volta para **6 em ~18 s** (medido 25/09) sem ninguém intervir: o `selfHeal: true` dispara um sync automático e re replica o valor do Git.

**Não** use `kubectl scale` aqui: as réplicas de todos os Deployments de aplicação são ignoradas de propósito ( `ignoreDifferences + RespectIgnoreDifferences=true` ), porque quem manda nelas é o HPA. Escalar o ngo para 1 e vê-lo voltar a 2 prova o `minReplicas` do HPA, não o GitOps (era o teste antigo, corrigido em 25/09).

☐ Validado Responsável \_\_\_\_\_ Data/hora \_\_\_\_\_

**F0-08** Prometheus, Grafana, Loki e OpenTelemetry rodando

MEDIDO

Onde: **TERMINAL** **NAVEGADOR** Tempo: 6 min

- 1 Componentes da plataforma:

```
kubectl -n monitoring get pods
kubectl -n monitoring get deploy,ds -o name | grep -i otel
```

- 2 Alvos do Prometheus. Deixe este comando aberto num segundo terminal:

```
kubectl -n monitoring port-forward svc/kube-prometheus-stack-prometheus 9090:9090
```

e abra `http://localhost:9090/targets`.

- 3 Loki: Grafana ( `$BASE/grafana/` ) → **Explore** → datasource **Loki** → consulta:

```
{namespace="solidary-donation"} | json | trace_id != ""
```

**RESULTADO ESPERADO**

Todos os pods **Running**. Dois coletores OpenTelemetry: `deployment/otel-collector` (gateway de traces e métricas) e `daemonset/...-agent` (logs). Prometheus: **27 alvos, 0 down**. Loki devolve logs em JSON com `trace_id`, que é a correlação log ↔ trace.

☐ Validado Responsável \_\_\_\_\_ Data/hora \_\_\_\_\_

Onde: **TERMINAL** **NAVEGADOR** Tempo: 8 min Print: **f0-trace-distribuido.png**

- 1 Prova de **entrega**, não só de envio: chave validada e nenhuma recusa:

```
kubectl -n monitoring logs deploy/otel-collector | grep -c "API key validation successful"
# >= 1
kubectl -n monitoring logs deploy/otel-collector | grep -c "403"
# 0
```

- 2 Volume de spans enviados. Deixe o port-forward aberto num segundo terminal:

```
kubectl -n monitoring port-forward deploy/otel-collector 8888:8888
curl -s http://localhost:8888/metrics | grep 'otelcol_exporter_sent_spans{.*datadog}'
```

- 3 Abra [app.datadoghq.com](https://app.datadoghq.com) (site **US1**, conta do grupo) → **APM** → **Services**.

- 4 **APM** → **Traces** → filtre **service:donation-service** → abra um trace de **POST /donations** (print **f0-trace-distribuido.png**).

- 5 **APM** → **Service Map**: dependências entre os serviços e a fila.

#### RESULTADO ESPERADO

Validação  $\geq 1$ , **0 x 403**, contador de spans subindo (25/09: 55.044). No Datadog: **donation-service**, **ngo-service** e **volunteer-service** com requisições, erros e latência; um trace que atravessa **donation-service** → **SQS** → **volunteer-worker**.

#### SE NÃO BATER

Serviços sem dados: rode P-04 e aguarde 2 min. Com 403: a chave ou o site estão errados. Quem tem acesso à chave roda **./solidary datadog** (entrada oculta, grava no cofre e reinicia o coletor).

☐ Validado Responsável \_\_\_\_\_ Data/hora \_\_\_\_\_

Onde: **TERMINAL** Tempo: 5 min Caminho: NLB → ingress-nginx → Service → pod → banco

1 **ngo-service** (Flask + RDS). Cadastre uma ONG e teste a validação:

```
curl -s -X POST $BASE/ngo/ngos -H 'Content-Type: application/json' -d '{"name":"Instituto Semear","email":"contato-`${date +%s}`@semear.org","cause":"Educacao","city":"Curitiba"}'
curl -s -o /dev/null -w '%{http_code}\n' -X POST $BASE/ngo/ngos -H 'Content-Type: application/json' -d '{"name":"X","email":"nao-e-email","cause":"Y","city":"Z"}'
```

2 **donation-service** (Go, hot path). Doação válida e inválida:

```
curl -s -o /dev/null -w '%{http_code}\n' -X POST $BASE/donations -H 'Content-Type: application/json' -d '{"ngo_id":1,"amount":150.50,"donor_name":"Maria Silva"}'
curl -s -o /dev/null -w '%{http_code}\n' -X POST $BASE/donations -H 'Content-Type: application/json' -d '{"ngo_id":0,"amount":-5,"donor_name":""}'
```

3 **volunteer-service** (Flask + DynamoDB):

```
curl -s -o /dev/null -w '%{http_code}\n' -X POST $BASE/volunteers -H 'Content-Type: application/json' -d '{"ngo_id":1,"name":"Joao Souza","email":"joao@exemplo.org","skills":["logistica"]}'
curl -s $BASE/volunteers/1
```

4 **Fluxo assíncrono**: a doação do passo 2 gera um evento na SQS, consumido pelo worker:

```
kubectl -n solidarity-volunteer logs -l app=volunteer-worker --prefix --since=5m | tail -5
aws sqs get-queue-attributes --queue-url $(aws sqs get-queue-url --queue-name solidaritytech-prod-donation-events --query QueueUrl --output text) --attribute-names ApproximateNumberOfMessages RedrivePolicy
```

**RESULTADO ESPERADO**

ONG: JSON com `id` (201); e-mail inválido: **400**. Doação: **201**; inválida: **400**. Voluntário: **201**; listagem: **200**. O worker registra o processamento ( `-l app=volunteer-worker --prefix` lê **todas** as réplicas: `logs deploy/...` lê só uma e, com 6 réplicas, costuma não mostrar a doação); fila com `ApproximateNumberOfMessages = 0` (drenada) e DLQ com `maxReceiveCount = 3`.

**SE NÃO BATER**

`HTTP 000` = timeout, não erro da aplicação: confira `$BASE` (sessão nova, NLB novo) e se o DNS do NLB já propagou (até 3 min após a subida).

☐ Validado Responsável \_\_\_\_\_ Data/hora \_\_\_\_\_

Onde: **TERMINAL** Tempo: 6 min (durante P-04)

- 1 Com a carga rodando, acompanhe os HPAs e o worker:

```
kubectl get hpa -A  
kubectl -n solidary-volunteer get pods -l app=volunteer-worker -w
```

- 2 Confirme que o metrics-server responde:

```
kubectl top nodes
```

**RESULTADO ESPERADO**

4 HPAs, nenhum com `<unknown>`. O `volunteer-worker` escala de **1 para 6** réplicas no pico e volta a **1** cerca de 10 min depois da carga, a janela de estabilização de 600 s do HPA (medido em 25/09: 13:03 → 6, 13:14 → 1). Parado, a CPU do worker fica abaixo de 15% do request.

☐ Validado Responsável \_\_\_\_\_ Data/hora \_\_\_\_\_

# F1 SRE: SLI, SLO e MTTR

Referência: docs/03-sre/sli-slo-sla.md

| SLI (donation-service) | Golden Metric | SLO                       | Recording rule                                     | 25/09         |
|------------------------|---------------|---------------------------|----------------------------------------------------|---------------|
| Disponibilidade        | Erros         | 99,9 %<br>(janela 7 dias) | slo:donation_disponibilidade_erro:ratio_rate5m     | 0 erro        |
| Latência               | Latência      | 99 % < 300 ms             | slo:donation_latencia:p95                          | p95<br>4,8 ms |
| Frescor da fila        | Saturação     | 99,5 % < 60 s             | slo:donation_frescor_erro:ratio_rate1h             | 0 erro        |
| Error budget restante  | n/a           | n/a                       | slo:donation_disponibilidade:error_budget_restante | 100 %         |

## F1-01 No mínimo dois SLIs do donation-service, medidos

MEDIDO

Onde: NAVEGADOR Tempo: 4 min Pré-requisito: P-04 e port-forward do F0-08

- 1 Em `http://localhost:9090/graph`, execute cada uma das quatro *recording rules* da tabela acima.
- 2 Em `http://localhost:9090/rules`, confira os grupos `solidarytech.slo.*`.

### RESULTADO ESPERADO

As quatro consultas retornam número (não vazio, não NaN). Regras com estado OK.

☐ Validado Responsável \_\_\_\_\_ Data/hora \_\_\_\_\_

## F1-02 SLO definido para cada SLI

DOCUMENTAL

Onde: REPOSITÓRIO Tempo: 3 min

- 1 Leia `docs/03-sre/sli-slo-sla.md`: cada SLI tem SLO, janela, justificativa e SLA contratual derivado.
- 2 Confira que o código implementa o que o documento promete:

```
grep -n "record:" gitops/addons/observabilidade-config/slo-rules.yaml
```

### RESULTADO ESPERADO

Três SLOs (99,9 %, 99 % < 300 ms, 99,5 % < 60 s), e as *recording rules* do arquivo batem com os nomes da tabela.

`./solidary check` valida esse contrato PromQL × código automaticamente.

☐ Validado Responsável \_\_\_\_\_ Data/hora \_\_\_\_\_

**F1-03 Dashboard exclusivo de SLOs e consumo do error budget****MEDIDO**Onde: **NAVEGADOR** Tempo: 3 min Print: **f1-dash-board-sre.png**

- 1 Grafana ( `$BASE/grafana/` ) → **Dashboards** → **SolidaryTech — SRE: SLOs e Error Budget**.
- 2 Ajuste o intervalo para **Last 1 hour** e capture o print com a carga recém-executada.

**RESULTADO ESPERADO**

Painéis *Disponibilidade (SLO 99,9%)*, *Latência < 300ms (SLO 99%)* e *Frescor da fila < 60s (SLO 99,5%)* com valores, e o error budget restante em ~100 %.

**SE NÃO BATER**

Painéis com `NaN` ou "No data": não há tráfego na janela. Rode P-04.

☐ Validado Responsável \_\_\_\_\_ Data/hora \_\_\_\_\_**F1-04 Evidenciar como a stack reduz o MTTR: chaos drill****MEDIDO**Onde: **TERMINAL** **NAVEGADOR** Tempo: 10 min Roteiro completo: **docs/03-sre/mttr-chaos-drill.md**

**Altera produção por ~2 min, de propósito:** o `donation-service` fica sem réplicas. O HPA não recria a partir de zero e o ArgoCD ignora réplicas deste serviço, então a falha persiste até o passo 4. Não pule o passo 4.

- 1 Anote a linha de base (hora e pods):

```
date -u +%T; kubectl -n solidary-donation get pods
```
- 2 Provoque a falha e anote `T_falha` :

```
kubectl -n solidary-donation scale deployment/donation-service --replicas=0; date -u +%T
```
- 3 Observe sem intervir: Prometheus ( `localhost:9090/alerts` ) até o alerta `DeploymentSemReplicaDisponivel` ficar **Firing**; anote `T_alerta` . No Grafana SRE, veja o error budget cair.
- 4 Restaure e anote `T_recuperado` :

```
kubectl -n solidary-donation scale deployment/donation-service --replicas=2
kubectl -n solidary-donation rollout status deployment/donation-service; date -u +%T
```

**RESULTADO ESPERADO**

Deteção automática (MTTD) na casa de **76 s**, contra ~6 h na linha de base da fase anterior (usuário reclamando). Preencha a tabela de resultados do `mttr-chaos-drill.md` ; o documento rotula o que é medido e o que é meta.

☐ Validado Responsável \_\_\_\_\_ Data/hora \_\_\_\_\_

**F2-01 Tags obrigatórias no Terraform em todos os recursos**

MEDIDO

`Project=SolidaryTech, Environment=Production, CostCenter=NGO-Core`.Onde: `TERMINAL` `CONSOLE AWS` Tempo: 4 min Print: `f2-tags-console.png`

- 1 Conte os recursos com cada tag:

```
for t in CostCenter=NGO-Core Project=SolidaryTech Environment=Production; do k=${t%*=*};  
v=${t#*=*}; echo "$t: $(aws resourcegroupstaggingapi get-resources --tag-filters  
Key=$k,Values=$v --query 'length(ResourceTagMappingList)')"; done
```

- 2 Confira a origem no código (`default_tags` do provider):

```
grep -nE "default_tags|Project +=|Environment +=|CostCenter +=\" infra/environments/prod-  
use1/providers.tf
```

- 3 Console AWS → **Resource Groups & Tag Editor** → **Tag Editor** → região `us-east-1`, tipo *All supported*, tag `CostCenter = NGO-Core` → **Search** (print `f2-tags-console.png`).

**RESULTADO ESPERADO**

`default_tags` com `Project = "SolidaryTech"`, `Environment = "Production"`, `CostCenter = "NGO-Core"`. **O mesmo número nas três tags** (42 a 45: o total oscila porque inclui recursos derivados que herdam as tags, como nós, ENIs, discos e snapshot automático do RDS). A seção I do `./solidary evidencias` mostra **0** recursos com `Environment ≠ Production`.

☐ Validado Responsável \_\_\_\_\_ Data/hora \_\_\_\_\_

**F2-02 Rightsizing de requests/limits nos YAML, via GitOps**

MEDIDO

Onde: `TERMINAL` Tempo: 4 min Evidência: `07-evidencias/rightsizing-medido.txt`

- 1 Requests e limits efetivamente aplicados:

```
kubectl get deploy -A -o custom-  
columns='NS:.metadata.namespace,APP:.metadata.name,REQ_CPU:.spec.template.spec.containers[0].  
resources.requests.cpu,LIM_CPU:.spec.template.spec.containers[0].resources.limits.cpu,REQ_MEM  
:.spec.template.spec.containers[0].resources.requests.memory,LIM_MEM:.spec.template.spec.cont  
ainers[0].resources.limits.memory' | grep -E "NS|solidary"
```

- 2 Uso real sob carga, para comparar com o reservado:

```
kubectl top pods -A | grep solidary
```

- 3 Mostre que o ajuste veio do Git (e não do cluster):

```
git log --oneline -- gitops/apps | grep -iE "rightsiz|recurso|limits" | head -5
```

**RESULTADO ESPERADO**

Todos os Deployments das aplicações com os quatro valores preenchidos; uso real abaixo dos limites e coerente com a tabela antes/depois do `rightsizing-medido.txt`.

☐ Validado Responsável \_\_\_\_\_ Data/hora \_\_\_\_\_

**F2-03 Forecast mensal e custo por namespace**

DOCUMENTAL

Onde: REPOSITÓRIO NAVEGADOR Tempo: 4 min Print: f2-dashboard-finops.png

- 1 Leia docs/04-finops/README.md §3: forecast item a item.
- 2 Grafana → **SolidaryTech — FinOps: custo e rightsizing** (dados do OpenCost), para o print f2-dashboard-finops.png.

**RESULTADO ESPERADO**Forecast de **US\$ 202,74/mês** com memória de cálculo. Painel FinOps com custo por namespace.☐ Validado Responsável \_\_\_\_\_ Data/hora \_\_\_\_\_**F2-04 Pelo menos uma recomendação nativa de nuvem**

DOCUMENTAL

Onde: REPOSITÓRIO Tempo: 2 min

- 1 Leia docs/04-finops/README.md §4.

**RESULTADO ESPERADO****5 recomendações** quantificadas (economia estimada em US\$/mês), cada uma com o serviço AWS nativo correspondente.☐ Validado Responsável \_\_\_\_\_ Data/hora \_\_\_\_\_

**F3-01 Ativar a IA do APM (Watchdog) para detectar anomalias****PENDENTE DO GRUPO**

Onde: NAVEGADOR Tempo: 10 min Print: f3-anomalia-watchdog.png

A base técnica já está comprovada: as trace metrics do `datadog/connector` chegam ao Datadog, e é sobre elas que o Watchdog trabalha. Falta criar o monitor, o que exige a conta do grupo.

- 1 Rode P-04 (carga) para haver tráfego.
- 2 `app.datadoghq.com` → **Monitors** → **New Monitor** → tipo **Watchdog**.
- 3 Em *Define the alert*: categoria **APM**; escopo `env:prod`; serviços `donation-service`, `ngo-service`, `volunteer-service`.
- 4 Em *Notify your team*: nome "[SolidaryTech] Anomalia detectada pelo Watchdog"; destinatário = canal do ChatOps ou o e-mail do grupo → **Create**.
- 5 Abra **Watchdog** no menu lateral e capture o print (anomalias ou a lista de serviços monitorados).

**RESULTADO ESPERADO**

Monitor Watchdog criado e ativo para `env:prod`. O Watchdog aprende a linha de base antes de acusar: se nenhuma anomalia surgir no dia, o print do monitor configurado mais a página do Watchdog com os serviços demonstra a funcionalidade ativa.

☐ Validado Responsável \_\_\_\_\_ Data/hora \_\_\_\_\_**F3-02 Ciclo de vida do incidente: detecção → post-mortem → stakeholders****DOCUMENTAL**

Onde: REPOSITÓRIO Tempo: 4 min

- 1 Diagrama: `docs/relatorio/img/ciclo-incidente.svg` (também no PDF do relatório).
- 2 Processo: `docs/05-itsm-aiops/README.md` (severidades, papéis, comunicação).
- 3 Exercício real: `docs/05-itsm-aiops/post-mortem-2026-09-10-frescor.md`.

**RESULTADO ESPERADO**

Todas as etapas desenhadas (detecção, triagem, mitigação, resolução, post-mortem, comunicação a stakeholders) e um post-mortem real, sem culpados e com ações corretivas.

☐ Validado Responsável \_\_\_\_\_ Data/hora \_\_\_\_\_

## F4 Segurança, Backup e Recuperação de Desastres

Referências: docs/06-dr-pcn/pcn.md

· ADR-014

### F4-01 PCN executivo com RTO e RPO para os dados de doação

MEDIDO

Onde: REPOSITÓRIO TERMINAL Tempo: 4 min

- 1 Leia docs/06-dr-pcn/pcn.md : RTO, RPO e justificativa de negócio.
- 2 Comprove que o RPO de 15 min é sustentado pelo RDS (backup contínuo, PITR):

```
aws rds describe-db-instances --db-instance-identifier solidaritytech-prod-postgres --query "DBInstances[0].{retencao:BackupRetentionPeriod,ultimoRestauravel:LatestRestorableTime,publico:PubliclyAccessible,cifrado:StorageEncrypted}" --output table
```

#### RESULTADO ESPERADO

**RTO 1 h · RPO 15 min.** RDS com retenção de **7 dias**, LatestRestorableTime a poucos minutos do horário atual (bem dentro do RPO), `publico = False`, `cifrado = True`.

☐ Validado Responsável \_\_\_\_\_ Data/hora \_\_\_\_\_

### F4-02 Opção A: Velero com backup de manifestos e volumes para bucket externo

MEDIDO

Onde: TERMINAL Tempo: 4 min Print: f4-velero-backups.png

- 1 Agendamentos e backups (o segundo comando é o print):

```
kubectl -n velero get schedules.velero.io
kubectl -n velero get backups.velero.io
```

- 2 O armazenamento está autenticado (sem IRSA, pelo perfil do nó):

```
kubectl -n velero get backupstoragelocations
```

- 3 O dado saiu do cluster e da região:

```
VB=$(aws s3api list-buckets --query "Buckets[?starts_with(Name,'solidaritytech-prod-velero')].Name" --output text)
aws s3api get-bucket-location --bucket $VB --output text
aws s3 ls s3://$VB/backups/ --recursive --human-readable | tail -5
```

#### RESULTADO ESPERADO

Schedules `velero-diario` (03:00, com volumes, 30 dias) e `velero-horario-aplicacoes` (de hora em hora). Backups `Completed` (25/09: 914/914 itens, 3/3 snapshots de volume). BSL `Available`. Bucket em **us-west-2** com o `.tar.gz` dos manifestos.

#### SE NÃO BATER

Nenhum backup listado numa sessão recém-criada: o horário ainda não chegou. Dispare um backup a partir do schedule diário ( docs/10 , passo 13.2) e aguarde ~15 s.

☐ Validado Responsável \_\_\_\_\_ Data/hora \_\_\_\_\_

Onde: **TERMINAL** Tempo: 5 min Evidência: **07-evidencias/dr-velero-backup-restore.txt**

**Duas travas obrigatórias**, aprendidas em execução real (item 6 da evidência): (1) só restaure com **todos** os snapshots do backup em **completed**; (2) só apague PV cujo nome comece com **velero-clone-** e cujo **claimRef** esteja em **dr-drill-restore**. Nunca apague um PV **pvc-...**: esse é de produção.

1 Escolha o backup e confirme os snapshots:

```
BK=$(kubectl -n velero get backups.velero.io --sort-by=.metadata.creationTimestamp -o
jsonpath='{.items[-1:].metadata.name}'); echo $BK
aws ec2 describe-snapshots --owner-ids self --filters "Name=tag:velero.io/backup,Values=$BK"
--query 'Snapshots[].[SnapshotId,State]' --output text # todos "completed"?
```

2 Restaure só o PVC do Prometheus num namespace isolado:

```
cat <<EOF | kubectl apply -f -
apiVersion: velero.io/v1
kind: Restore
metadata: { name: drill-$(date +%H%M%S), namespace: velero }
spec:
  backupName: $BK
  includedNamespaces: ["monitoring"]
  includedResources: ["persistentvolumeclaims","persistentvolumes"]
  labelSelector: { matchLabels: { app.kubernetes.io/name: prometheus } }
  namespaceMapping: { monitoring: dr-drill-restore }
  restorePVs: true
EOF
kubectl -n dr-drill-restore get pvc -w # Ctrl+C em Bound
```

3 Confira as travas e limpe só o que é do drill:

```
PV=$(kubectl -n dr-drill-restore get pvc -o jsonpath='{.items[0].spec.volumeName}'); echo $PV
# TEM de ser velero-clone-...
kubectl delete namespace dr-drill-restore
case "$PV" in velero-clone-*) kubectl delete pv "$PV" --ignore-not-found ;; *) echo "NAO
APAGAR: $PV" ;; esac
```

**RESULTADO ESPERADO**

Restore **Completed**; PVC **Bound em ~12 s** num PV **velero-clone-...** criado **a partir do snapshot**; depois da limpeza, namespace e PV do drill **NotFound**, sem custo órfão.

☐ Validado Responsável \_\_\_\_\_ Data/hora \_\_\_\_\_

F4-04

## Opção B: ambiente espelho em outra região com um comando

MEDIDO

Onde: **TERMINAL** Tempo: 4 min Print: **f4-dr-plan.png**

- 1 Plano (somente leitura, não cria nada) da região secundária, com os mesmos módulos:

```
AMBIENTE=dr-usw2 ./solidary plan
```

## RESULTADO ESPERADO

Plan: 34 to add, 0 to change, 0 to destroy. em us-west-2: EKS, RDS, DynamoDB, SQS+DLQ e rede, com Environment=Production e Role=warm-standby. Só leitura da LabRole : 0 recurso IAM. Subir de fato: ./solidary dr-up (não execute sem necessidade: gera custo).

☐ Validado Responsável \_\_\_\_\_ Data/hora \_\_\_\_\_

F4-05

## Segurança de rede: isolamento entre serviços e bloqueio do IMDS

MEDIDO

Onde: **TERMINAL** Tempo: 4 min

- 1 Políticas de rede aplicadas:

```
kubectl get networkpolicies -A
```

- 2 Conexão não autorizada é negada (ngo não fala com donation):

```
kubectl -n solidary-ngo run teste-np --rm -i --image=curlimages/curl:8.11.1 --restart=Never -  
- curl -s -o /dev/null -w '%{http_code}\n' -m 8 http://donation-service.solidary-  
donation.svc.cluster.local/donations
```

- 3 A API pública (sem autenticação) não alcança a credencial da nuvem (IMDS) nem a internet:

```
kubectl -n solidary-ngo exec deploy/ngo-service -- python -c "import  
urllib.request;urllib.request.urlopen('http://169.254.169.254/latest/meta-data/',timeout=3)"  
kubectl -n solidary-ngo exec deploy/ngo-service -- python -c "import  
urllib.request;urllib.request.urlopen('https://www.google.com',timeout=3)"
```

- 4 Banco privado e bucket sem acesso público:

```
aws s3api get-public-access-block --bucket $VB --query PublicAccessBlockConfiguration --  
output text
```

## RESULTADO ESPERADO

11 NetworkPolicies. Teste do passo 2: 000 após 8 s (pacote descartado). IMDS: URLError: timed out ; internet: Network is unreachable. Bucket: True True True True. RDS PubliclyAccessible = False (F4-01).

☐ Validado Responsável \_\_\_\_\_ Data/hora \_\_\_\_\_

**F4-06** Gestão de segredos: nenhum segredo no Git

MEDIDO

Onde: **TERMINAL** Tempo: 3 min Decisão: ADR-014

- 1 Segredos no cofre (só os nomes, nunca o valor):

```
aws secretsmanager list-secrets --query 'SecretList[?contains(Name,`solidarytech`)].Name' --output text
```

- 2 Nenhuma senha literal nos manifestos:

```
grep -rniE 'password:[[:space:]]*[^$\\[[:space:]]' gitops/ | grep -v 'existingSecret\\|secretKeyRef\\|passwordKey' || echo "nenhuma senha no gitops/"
```

- 3 A credencial do APM chega ao cluster pelo cofre (metadados, sem o valor):

```
kubectl -n monitoring get secret apm-credentials -o jsonpath='{.metadata.labels}{\\n"}{.metadata.annotations}{\\n"}'
```

**RESULTADO ESPERADO**

`solidarytech/datadog` e o segredo do RDS listados; o `grep` retorna "nenhuma senha no gitops/"; o Secret traz `solidarytech.io/origem: secretsmanager` e `solidarytech.io/cofre: solidarytech/datadog` com a versão do cofre, sem o valor.

☐ Validado Responsável \_\_\_\_\_ Data/hora \_\_\_\_\_

**F4-07** Conformidade com as restrições do AWS Academy

MEDIDO

Onde: **TERMINAL** Tempo: 3 min

- 1 Gates locais (os mesmos da CI):

```
./solidary check
```

- 2 Nenhum recurso IAM criado; a LabRole é apenas lida:

```
terraform -chdir=infra/environments/prod-use1 state list | grep -E '(^|\\.)aws_iam_' | grep -v '^data\\.' | grep -v '\\.data\\. ' || echo "nenhum recurso IAM"
./solidary conformidade
```

**RESULTADO ESPERADO**

"Todos os gates passaram." (política do Academy com 16 verificações, observabilidade, contrato PromQL, workflows, manifestos, `terraform fmt`, links). **"nenhum recurso IAM"**. Relatório de conformidade: regiões permitidas, instâncias ≤ `large`, On-Demand.

☐ Validado Responsável \_\_\_\_\_ Data/hora \_\_\_\_\_

### E-01 Repositório público com IaC, manifestos e pipelines

MEDIDO

- 1 Abra uma **janela anônima** e acesse [github.com/DemarchiWorking/fiap-tc-5-solidarytech](https://github.com/DemarchiWorking/fiap-tc-5-solidarytech) (é o que a banca verá).
- 2 Confira as pastas `infra/`, `gitops/apps/`, `.github/workflows/` e o workflow *Validação* verde no último commit.

#### RESULTADO ESPERADO

Repositório acessível sem login; último commit com *Validação* verde.

☐ Validado    Responsável \_\_\_\_\_    Data/hora \_\_\_\_\_

**E-02 Evidências visuais: os 10 prints do relatório****PENDENTE DO GRUPO**

Salve em `docs/07-evidencias/` com **exatamente** estes nomes; o gerador do PDF os inclui sozinho e marca em vermelho o que faltar. Rode P-04 antes.

| Arquivo                                                        | Origem                                                       | Cartão |
|----------------------------------------------------------------|--------------------------------------------------------------|--------|
| <input type="checkbox"/> <code>f0-argocd.png</code>            | <code>\$BASE/argocd/</code> : 15 Applications Synced/Healthy | F0-07  |
| <input type="checkbox"/> <code>f0-pipeline-verde.png</code>    | GitHub Actions → CI — donation-service (5 jobs verdes)       | F0-04  |
| <input type="checkbox"/> <code>f0-pods-running.png</code>      | <code>kubectl get pods -A   grep solidary</code>             | F0-02  |
| <input type="checkbox"/> <code>f0-trace-distribuido.png</code> | Datadog → APM → Traces                                       | F0-09  |
| <input type="checkbox"/> <code>f1-dashboard-sre.png</code>     | Grafana → SRE: SLOs e Error Budget                           | F1-03  |
| <input type="checkbox"/> <code>f2-tags-console.png</code>      | AWS → Tag Editor → <code>CostCenter = NGO-Core</code>        | F2-01  |
| <input type="checkbox"/> <code>f2-dashboard-finops.png</code>  | Grafana → FinOps: custo e rightsizing                        | F2-03  |
| <input type="checkbox"/> <code>f3-anomalia-watchdog.png</code> | Datadog → Watchdog                                           | F3-01  |
| <input type="checkbox"/> <code>f4-velero-backups.png</code>    | <code>kubectl -n velero get backups.velero.io</code>         | F4-02  |
| <input type="checkbox"/> <code>f4-dr-plan.png</code>           | <code>AMBIENTE=dr-usw2 ./solidary plan</code>                | F4-04  |

Antes de cada print, confira que não aparece senha, token ou chave na tela (terminal com `./solidary senhas` aberto, abas do console AWS com credenciais).

**E-03 Vídeo (até 20 min) e PDF final****PENDENTE DO GRUPO**

- 1 Grave seguindo `docs/roteiro-video.md`, cobrindo: pitch; pipelines e ArgoCD (F0-04, F0-07 com o selfHeal); Terraform (F0-02); traces e alertas (F0-09, F1-04); dashboard SRE (F1-03); backup/DR (F4-02 a F4-04). Publique no YouTube como **não listado**.
- 2 Cole o link na seção 1 de `docs/relatorio/RELATORIO-DE-ENTREGA.md` e gere o PDF pelo PowerShell ou Git Bash do Windows:

```
python scripts/gerar-relatorio.py
```

- 3 Confira a rubrica e publique:

```
./solidary rubrica  
git add docs/ && git commit -m "docs: prints, video e PDF final" && git push
```

**RESULTADO ESPERADO**

PDF sem aviso vermelho no topo; `./solidary rubrica` sem pendências; o vídeo abre numa aba anônima.

☐ Validado Responsável \_\_\_\_\_ Data/hora \_\_\_\_\_

## Z-01 Registrar a evidência consolidada e desligar o ambiente

MEDIDO

- 1 Consolide tudo num arquivo versionado (somente leitura no ambiente):

```
./solidary evidencias # gera docs/07-evidencias/validacao-final.txt
```

- 2 Destrua o ambiente (o bucket de state e o cofre do Datadog são preservados):

```
./solidary lab-down
```

### RESULTADO ESPERADO

`validacao-final.txt` com as seções A a J preenchidas. `Destroy complete!` Custo evitado: ~US\$ 6,73/dia.

O `lab-down` não remove os volumes EBS criados pelos PVCs nem os snapshots do Velero. Confira no console (EC2 → Volumes / Snapshots) se sobrou algo com a tag `Project=SolidaryTech`.

☐ Validado Responsável \_\_\_\_\_ Data/hora \_\_\_\_\_

## Checklist de higiene antes da submissão

| Item                                                                                                    | Como                                                                               |
|---------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| <input type="checkbox"/> Rotacionar a chave do Datadog (trafegou em texto na fase anterior)             | Datadog → Organization Settings → API Keys; depois <code>./solidary datadog</code> |
| <input type="checkbox"/> Nenhuma chave <code>.pem</code> / <code>.ppk</code> dentro da pasta do projeto | <code>git status --ignored   grep -iE "pem ppk"</code>                             |
| <input type="checkbox"/> Nenhum segredo no histórico do Git                                             | Workflow <i>Terraform</i> , passo <i>gitleaks</i> , verde                          |
| <input type="checkbox"/> Repositório público                                                            | Teste em janela anônima (E-01)                                                     |
| <input type="checkbox"/> Ambientes de contas anteriores do lab desligados                               | <code>lab-down</code> em cada conta ainda acessível                                |

## A.1 Diagnóstico rápido

| Sintoma                                                       | Causa                                                                                         | Ação                                                                              |
|---------------------------------------------------------------|-----------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| <code>curl tcp [::1]:8080</code>                              | Terminal errado (PowerShell/CMD)                                                              | <code>WSL</code> → <code>tc5</code>                                               |
| <code>ExpiredToken</code> / <code>InvalidClientTokenId</code> | Sessão do lab vencida                                                                         | Refazer P-01 e P-02                                                               |
| <code>curl</code> devolve <code>000</code>                    | Timeout: <code>\$BASE</code> antigo ou DNS do NLB propagando                                  | Recalcular <code>\$BASE</code> ; aguardar 3 min                                   |
| API do EKS responde <b>401</b> no navegador                   | Correto: a API exige token AWS                                                                | Usar <code>\$BASE/...</code> no navegador e <code>kubectl</code> no WSL           |
| Painéis com <code>NaN</code> / "No data"                      | Sem tráfego na janela                                                                         | P-04 ( <code>./solidary carga</code> )                                            |
| HPA com <code>&lt;unknown&gt;</code>                          | metrics-server fora                                                                           | <code>kubectl -n argocd get app metrics-server</code>                             |
| Datadog sem serviços                                          | Sem carga, ou chave/site errados (403)                                                        | P-04; checar F0-09 passo 1                                                        |
| Application <code>OutOfSync</code> por minutos                | Sincronização em curso após deploy                                                            | Aguardar; <b>Refresh</b> → <b>Hard refresh</b> na UI                              |
| Chrome: "O site não pode fazer uma conexão segura"            | ArgoCD e Grafana são servidos em HTTP pelo NLB (sem TLS no lab); o modo HTTPS do Chrome avisa | Conferir que o endereço é o <code>\$BASE</code> e clicar em <b>Ir para o site</b> |
| Pré-voo acusa conta diferente                                 | O Academy trocou a conta                                                                      | <code>COMO-SUBIR.md</code> (~35 min, sem mudar código)                            |

## A.2 Tabela de acessos

| Interface           | Endereço                                                                                        | Credencial                                          |
|---------------------|-------------------------------------------------------------------------------------------------|-----------------------------------------------------|
| APIs                | <code>\$BASE/ngo/ngos</code> · <code>\$BASE/donations</code> · <code>\$BASE/volunteers/1</code> | abertas                                             |
| ArgoCD              | <code>\$BASE/argocd/</code>                                                                     | <code>admin</code> / <code>./solidary senhas</code> |
| Grafana             | <code>\$BASE/grafana/</code>                                                                    | <code>admin</code> / <code>./solidary senhas</code> |
| Prometheus          | <code>localhost:9090</code> (port-forward)                                                      | sem autenticação, só local                          |
| Métricas do coletor | <code>localhost:8888</code> (port-forward)                                                      | só local                                            |
| Datadog             | app.datadoghq.com (US1)                                                                         | conta do grupo                                      |
| GitHub Actions      | github.com/DemarchiWorking/fiap-tc-5-solidarytech/actions                                       | conta GitHub                                        |
| Console AWS         | AWS Academy → Start Lab → AWS                                                                   | sessão do Learner Lab                               |

### A.3 Recursos na AWS

| Recurso        | Identificador                                                                                                                          |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------|
| Cluster EKS    | <code>solidarytech-prod-eks</code> (us-east-1, 1.34)                                                                                   |
| RDS PostgreSQL | <code>solidarytech-prod-postgres</code>                                                                                                |
| Fila e DLQ     | <code>solidarytech-prod-donation-events</code> · <code>...-dlq</code>                                                                  |
| DynamoDB       | <code>SolidaryTechVolunteers</code>                                                                                                    |
| ECR            | <code>solidarytech/{ngo,donation,volunteer}-service</code>                                                                             |
| Buckets        | <code>solidarytech-prod-velero-&lt;6 últimos dígitos da conta&gt;</code> (us-west-2) · <code>solidarytech-prod-loki-&lt;...&gt;</code> |
| Cofre          | <code>solidarytech/datadog</code> (Secrets Manager)                                                                                    |
| Conta em 25/09 | <code>716532857874</code> : confira a atual com <code>aws sts get-caller-identity</code>                                               |

### A.4 Termo de aceite da validação

Declaramos que os cartões marcados como **Validado** foram executados no ambiente indicado e que os resultados obtidos correspondem aos resultados esperados, ou que as divergências foram registradas abaixo.

| Data da sessão | Conta AWS | Cartões validados | Divergências registradas |
|----------------|-----------|-------------------|--------------------------|
| ___ / 39       |           |                   |                          |

Leonardo Alves Freitas · rm369434

Antonio Eduardo Silveira Demarchi · rm370045